SLICE 16 — RECON + RESEARCH: the low-THC beverage transaction limit

Status: RECON ONLY. No code changed in this slice. **Date:** 2026-09-03 **Repo state at recon:** `main` @ `288b918e` **Requested by:** Michael Lyman (owner), Greenway Marijuana, Port Orchard WA **Standing rule applied:** *do not guess, do not assume. we build from fact, not memory.*

---

0. Read this first — one correction to the request

Michael asked for this limit as:

> "*If the total thc per unit is 4mg or less, you can sell up to **200 oz** of them rather than the 72 oz limit for all other regular liquids.*"

The rule is real, it does apply to Greenway, and the 4 mg-per-unit trigger is exactly right. **The number is 200 milligrams of active delta-9 THC, not 200 ounces.** Four independent primary sources say so, quoted verbatim in §1. This is not a small difference. Two hundred ounces of a beverage would be roughly one and a half cases; 200 mg of THC at the 4 mg-per-unit ceiling is **fifty units maximum**, and it is a *tighter* practical ceiling on some carts than the 72 oz rule it replaces, not a looser one.

Because the whole slice is sized off this number, nothing gets built until Michael confirms he has seen this correction. Building to "200 oz" would have shipped a register that lets staff sell roughly fifty times the legal amount of low-dose beverage, with the compliance engine reporting green the entire time.

---

1. The authoritative sources

1.1 WAC 314-55-095(1)(d)(i) — recreational transaction limits

Retrieved 2026-09-03 from `https://app.leg.wa.gov/WAC/default.aspx?cite=314-55-095`. Current version: WSR 24-21-051, filed 10/9/24, **effective 1/7/25**. Verbatim:

> *(A) One ounce of useable cannabis;*
>
> *(B) Sixteen ounces of cannabis-infused product meant to be eaten or swallowed in solid form;*
>
> *(C) Seven grams of cannabis-infused extract or cannabis concentrate for inhalation;*
>
> *(D) Ten units of a cannabis-infused product otherwise taken into the body;*
>
> *(E) Seventy-two ounces of cannabis-infused product in liquid form for oral ingestion or applied topically to the skin, **unless the product is packaged in individual units containing no more than four milligrams of active delta-9 THC per unit**; and*
>
> *(F) **Two hundred mg of active delta-9 THC** within a cannabis-infused product in liquid form if the product is packaged in individual units containing no more than four milligrams of active delta-9 THC per unit.*

Three things this text settles, and they drive the whole design:

1. **The unit is milligrams of THC.** Subparagraph (F) says "two hundred mg of active delta-9 THC". It is a potency cap, not a volume cap.
2. **(E) and (F) are mutually exclusive, not additive.** (E) contains the word "unless". A product that meets the ≤4 mg/unit packaging condition is *carved out of* the 72 oz bucket and governed by (F) instead. It does not consume both.
3. **The trigger is how the product is PACKAGED, not what the customer drinks.** "packaged in individual units containing no more than four milligrams" is a property of the product on the shelf. This means the classification is a **product attribute we can determine at intake**, not something computed at the register from cart contents. That is a large simplification and it is the reason Fix B from Slice 15 (a per-product flag) is the right shape here too.

1.2 RCW 69.50.360(3) — the enabling statute

Retrieved 2026-09-03 from `https://app.leg.wa.gov/RCW/default.aspx?cite=69.50.360`. Subsection (3)(d) reads:

> *200 milligrams of THC within a cannabis-infused product in liquid form if the product is packaged in individual units containing no more than four milligrams of THC per unit.*

The statute agrees with the rule. Note the RCW says "THC" where the WAC says "active delta-9 THC" — the WAC is the more specific and more recent instrument, and it is the one the LCB enforces, so **active delta-9 THC** is the measure we implement.

1.3 SHB 1249, Chapter 9, Laws of 2024 — where this came from

Enrolled session law, effective June 6, 2024. This is the bill that created the category: it struck the "or" following (c) and inserted the new (d) quoted above. Retrieved from the WA Legislature enrolled-bill PDF. Relevance: it confirms this is a **recent addition** (mid-2024, with

the implementing WAC effective 1/7/25), which explains cleanly why the limit is absent from our engine — the engine was built in Slice 34 against the then-current rule set, and the four buckets were correct at the time. This is not rot; it is a statute that moved.

1.4 LCB CR-102 rule-making notice

Retrieved from `lcb.wa.gov`. The agency's own plain-language description of what it was adopting:

> *authorizing retailers to sell **200 milligrams of THC** within a cannabis-infused product in liquid form, to a retail customer, if the product is packaged in units containing no more than 4 milligrams of THC per unit.*

Fourth independent confirmation, this one from the regulator that would write the violation.

1.5 The medical parallel — WAC 314-55-095(2)(d)

Same page, same retrieval. Verbatim:

> *A single transaction by a retail store with a medical cannabis endorsement to a qualifying patient or designated provider who is entered into the medical cannabis database is limited to three ounces of useable cannabis, 48 ounces of cannabis-infused product meant to be eaten or swallowed in solid form, 21 grams of cannabis-infused extract or cannabis concentrate for inhalation, and 216 ounces of cannabis-infused product in liquid form meant to be eaten or swallowed, **and up to 200 mg of active delta-9 THC** within a cannabis-infused product in liquid form meant to be eaten or swallowed if product is packaged in individual units containing no more than four milligrams of active delta-9 THC per unit.*

This is the single most surprising fact in the research and it must not be missed: the medical figure is also 200 mg. Every other bucket triples for a DOH-database patient — 1→3 oz, 16→48 oz, 7→21 g, 72→216 oz. The low-THC beverage cap does **not** triple. It is 200 mg for a recreational customer and 200 mg for a medical patient, identically. Any developer writing this feature on autopilot will write `200 * 3` for the medical profile because that is the pattern of the other four buckets, and that would be a real over-sale on every medical transaction. The plan in §5 pins this with a dedicated regression test whose only job is to fail if someone "fixes" it later.

Also note the medical clause says "meant to be eaten or swallowed" where the recreational (E) says "for oral ingestion **or applied topically to the skin**". The recreational liquid bucket includes topicals; the medical one, read literally, does not. Our engine currently maps `topical` into `liquid_edible` for both profiles (anchor `sales-limits-core.ts:87`). That is a **pre-existing** discrepancy, it is conservative in the customer's favour on the medical side, and it is explicitly **out of scope** for this slice. Flagging it so it is on the record, not fixing it here.

1.6 A source that must NOT be used

`warules.elaws.us` mirrors WAC 314-55-095 but its copy was last updated **November 2016** and contains no (E)/(F) split at all. Anyone checking that mirror would conclude this rule does not exist. Only `app.leg.wa.gov` (WAC + RCW) and `lcb.wa.gov` are acceptable sources for limit work.

1.7 CCRS

Searched the repo for a CCRS sales-file writer that transmits per-transaction limit buckets: none exists (`src/lib/ccrs/` has no sales-detail module; see §3.9). CCRS sales reporting is line-level — product, quantity, price — and carries no limit-bucket field. **There is no CCRS schema change in this slice.** The limit is enforced at the point of sale and evidenced by our own `sales_limit_events` ledger (migration `0045_sales_limits.sql:53`), which is what an LCB enforcement officer would be shown. That ledger already records `buckets` as JSON, so a fifth bucket flows into it with no migration.

---

2. What the limit actually means in practice for Greenway

At the ≤4 mg/unit packaging ceiling, 200 mg ÷ 4 mg = **50 units**. That is the absolute maximum number of low-THC beverage units in one transaction, and only if every unit is exactly 4 mg. At 2 mg/unit it is 100 units. At 10 mg/unit the product is **not** in this category at all and falls back to the 72 oz rule.

The comparison Michael will care about: a 12 fl oz can, counted the way our engine counts liquids today, contributes roughly 355 g toward a 2,016 g (72 oz) ceiling — about **five and a half cans** before the register blocks. Under the correct (F) rule a 4 mg can is capped at 50 cans. So for genuine low-dose beverages the new rule is **dramatically more permissive** and Michael's instinct that he is currently over-blocking real sales is correct. The error was only in the unit, not in the direction. A customer buying a case of 4 mg seltzers is being refused today and should not be.

---

3. Codebase recon — every surface, with anchors

All line numbers verified against `main` @ `288b918e` on 2026-09-03. Anchors are given as `path:line` so this document does not drift.

3.1 The engine — `src/lib/compliance/sales-limits-core.ts` (599 lines)

This is the heart of the change. Everything else is a consumer.

Anchor	Symbol	What it is

:35	`import { STATUTORY_GRAMS_PER_OUNCE }`	28 g/oz, from the shared module
:40	`GRAMS_PER_OUNCE`	re-export
:43	`type LimitBucket`	**`"usable" | "solid_edible" | "concentrate" | "liquid_edible"` — four buckets, no fifth**
:45	`LIMIT_BUCKETS`	the readonly array driving every loop and every UI table
:52	`LIMIT_BUCKET_LABELS`	display strings
:60	`type LimitProfile`	**`{usable, solid_edible, concentrate, liquid_edible}` — every field a number of GRAMS**
:68	`RECREATIONAL_LIMITS`	`liquid_edible: 72 * GRAMS_PER_OUNCE` → 2016
:76	`MEDICAL_LIMITS`	`liquid_edible: 216 * GRAMS_PER_OUNCE` → 6048
:87	`categoryToBucket()`	maps `edible-liquid`, `tincture`, `topical` → `liquid_edible`
:140	`ALL_LIMIT_CATEGORY_SLUGS`	18 slugs
:162	`bucketCategories()`	groups slugs by bucket; **builds a literal 4-key object**
:189	`DEFAULT_UNIT_GRAMS`	`"edible-liquid": 28` — the per-unit fallback
:210	`type LimitOverrides`	`Partial<LimitProfile> & {unitGrams?}`
:216	`type LimitCartLine`	`{category, quantity, grams?}` — **no potency field**
:223	`type BucketUsage`	`{bucket, label, usedGrams, maxGrams, ratio, overBy, exceeded}` — **field names hard-code "Grams"**
:236	`type LimitEvaluation`	`{customerType, buckets, blocked, reasons, untrackedLines}`
:251	`gramsToOunces()`	round-3 divide
:265	`clampLimitProfile()`	AN-2 statutory clamp; **enumerates all four keys literally**
:285	`resolveLimits()`	**enumerates all four keys literally**
:302	`lineGrams()`	explicit `line.grams` wins, else `unitGrams[cat] ?? DEFAULT_UNIT_GRAMS[cat] ?? 0`
:315	`evaluateCart()`	**builds a literal 4-key `totals` object**; reasons string is hard-coded `"... oz exceeds the ... oz ... limit"`
:374	`__runSalesLimitTests()`	pure self-tests
:407	assertion	`categoryToBucket("edible-liquid") === "liquid_edible"`
:419	assertion	`RECREATIONAL_LIMITS.liquid_edible === 2016`
:589	assertion	`bc.liquid_edible.includes("edible-liquid")`

The architectural finding. `LimitProfile` (`:60`) types every bucket as a number of grams, and `BucketUsage` (`:223`) names its fields `usedGrams` / `maxGrams`. The new limit is denominated in **milligrams of THC**. There is no way to express it in the current type without either (a) lying about the unit and storing mg in a field called `usedGrams`, or (b) making the model unit-aware. Option (a) would silently corrupt every downstream consumer — the admin page at `src/app/admin/compliance/sales-limits/page.tsx:158` formats every non-concentrate bucket as `gramsToOunces(g) oz`, so a 200 mg cap would render to the owner as **"7.143 oz"**. Option (b) is the honest one and is what §5 proposes.

Also note four places enumerate the bucket keys **literally** rather than looping over `LIMIT_BUCKETS`: `bucketCategories()` `:162`, `clampLimitProfile()` `:265`, `resolveLimits()` `:285`, and the `totals` object inside `evaluateCart()` `:315`. Adding a bucket to the union at `:43` will therefore produce **compile errors at all four sites**, which is exactly what we want — TypeScript will refuse to let us forget one. The build plan leans on this deliberately.

Pre-existing doc drift found (not a bug, but worth one line). The file header at `:12–13` claims `solid_edible` = 453.592 g and `liquid_edible` = 2041.166 g. The actual code at `:70` and `:72` computes 16 × 28 = **448** and 72 × 28 = **2016**. The header comment was written against the 28.35 g/oz metric conversion and never updated when GW-016 standardised the enforcement engine on 28 g/oz. The **code is correct and conservative**; only the comment is stale. Cheap to fix while we are in the file.

3.2 The server wrapper — `src/lib/compliance/sales-limits.ts` (318 lines)

Anchor	Symbol	Note
:25–26	`export *`	re-exports the core + gate; every consumer imports through here
:28	`type SalesLimitSettings`	`rec` / `med` are **inline 4-field literals**, not `LimitProfile`
:38	`DEFAULT_SALES_LIMIT_SETTINGS`	`liquid_edible: 2016` / `6048`
:48	`type SettingsRow`	the 12 DB columns
:74	`getSalesLimitSettings()`	selects the column list at `:80`; **clamps on read** `:92` / `:101`
:116	`type SalesLimitSettingsInput`	inline 4-field literals again
:125	`updateSalesLimitSettings()`	**clamps on write** `:137–138`; column map `:144–151`
:162	`overridesFor()`	enumerates 4 keys literally
:181	`evaluateCartWithSettings()`	the website's soft-check entry point
:195	`logSalesLimitEvent()`	inserts into `sales_limit_events`; **`buckets` is JSON — no migration needed for a 5th**
:239	`listRecentSalesLimitOverrides()`	back-office audit panel feed
:283	`enforceSalesLimitForSale()`	**the authoritative register/completion gate**

3.3 The database — `supabase/migrations/0045_sales_limits.sql`

Anchor	What
:18	`create table sales_limit_settings` (singleton, `id boolean primary key`)
:25–28	`rec_usable_grams` 28, `rec_solid_grams` 448, `rec_concentrate_grams` 7, **`rec_liquid_grams` 2016**
:30–33	`med_*` 84 / 1344 / 21 / **6048**
:35	`unit_grams_json jsonb`

:53	`create table sales_limit_events`
:59	**`buckets jsonb`** — schemaless, absorbs a fifth bucket for free
:70–83	RLS: staff read settings, admin write, staff all on events

Note the `med_solid_grams` default here is **1344** (48 × 28) and matches `sales-limits.ts:42`, but `sales-limits-core.ts:78` computes `48 * GRAMS_PER_OUNCE` = 1344 while its own comment says "1360.8". Same stale-comment class as §3.1; harmless.

A **new migration is required** for two columns (`rec_low_thc_liquid_thc_mg`, `med_low_thc_liquid_thc_mg`) plus the per-product flag. Details in §5.

3.4 The register — `src/app/pos/SaleFlow.tsx` + `src/lib/pos/sale-flow-core.ts`

Anchor	Symbol	Note
`sale-flow-core.ts:34–37`	imports `LimitCartLine`, `LimitEvaluation`, `LimitProfile`	
`sale-flow-core.ts:51`	imports `lineGramsFromUnit`	AN-1 per-variant grams
`sale-flow-core.ts:69`	`category` on the POS card	"drives tax divisor + limit bucket"
`sale-flow-core.ts:80–82`	`unitGrams` on the card	parsed server-side from the variant label
`sale-flow-core.ts:109`	`type PosLimitSettings`	`{enforce, hardBlock, rec: LimitProfile, med: LimitProfile}`
`sale-flow-core.ts:121`	`limits` on the device bundle	**this is what ships to the iPad**
`sale-flow-core.ts:285`	`unitGrams` on the priced line	rides the sale payload
`sale-flow-core.ts:376`	`type LimitJudgement`	
`sale-flow-core.ts:384`	`judgeLimits()`	applies enforce/hardBlock semantics
`sale-flow-core.ts:406`	**`limitLinesFor()`**	**builds `{category, quantity, grams?}` — the exact function that must learn to carry potency**
`sale-flow-core.ts:655`	self-test settings	`liquid_edible: 2016 / 6048`
`SaleFlow.tsx:38`	imports `limitLinesFor`	
`SaleFlow.tsx:2374`	`judgeLimits(limitLinesFor(priced.lines), carded ? "medical" : "recreational", bundle.limits)`	**the live register meter**

3.5 The device bundle feed — `src/app/api/pos/menu/route.ts`

Anchor	What
:32	imports `gramsFromVariantLabel`
:161	`category: String(item.category)` on the card
:163	`variantLabel: variant.label || null`
:169	**`unitGrams: gramsFromVariantLabel(variant.label)`** — the only potency-adjacent field on the card
:181–182	`thc: item.thc`, `cbd: item.cbd` — **display strings, not numbers**
:260	`unitGrams: limitSettings.unitGrams` in the bundle

This is where the new per-unit-mg fact must be attached to the card. Everything the register knows about a product arrives through this route.

3.6 The customer website — the hard blocks Michael mentioned

Anchor	What
`src/lib/menu/cart-limit-meter-core.ts:43`	`type CartLimitLineInput = {category, quantity, variantLabel}` — **the three fields the website passes**
:57	`NEAR_LIMIT_RATIO = 0.8`
:65	`cartLimitLines()` — mirrors `limitLinesFor`
:82	`evaluateCartMeter()`
:91	`meterStatus()` → `ok | near | over`
:98	`activeBuckets()` — filters to `usedGrams > 0`
:122	**`cartLimitBlock()`** — the function both hard blocks call
:134	`__runCartLimitMeterCoreTests()`
`src/components/cart/CartLimitMeter.tsx:43`	the meter component
:70	`buckets.map(...)` — renders a bar per active bucket
`src/components/cart/CartProvider.tsx:549`	**`const {over: overLimit} = cartLimitBlock(items)`**
:599	`<CartLimitMeter items={items} />` in the drawer
:66	`type CartItemInput` — **has `category`, `variantLabel`, `quantity`; NO potency field**
`src/components/checkout/CheckoutFlow.tsx:71`	**`const {over: overLimit} = useMemo(() => cartLimitBlock(items), [items])`**
:339	`<CartLimitMeter items={items} />` on the checkout page

Both website hard blocks (cart drawer "Proceed to Checkout", checkout "Place Order") route through the **same** `cartLimitBlock()` at `cart-limit-meter-core.ts:122`. Fix that one function and both blocks are fixed. `CartItemInput` at `CartProvider.tsx:66` must gain the potency field, and whatever adds items to the cart must supply it.

3.7 The server order gate — `src/app/api/orders/route.ts`

Anchor	What
`:27`	imports `evaluateCartWithSettings`, `logSalesLimitEvent`
`:117`	`evaluateCartWithSettings(repriced.limitLines, "recreational")` — **soft check at placement**
`:119–122`	sets `limitFlag` / `limitReasons`, logs the event
`:143`, `:252`	flags persisted on the order

3.8 The completion gate — `src/lib/orders/completion-gate.ts`

Anchor	What
`:21`	doc: "S-1b sales-limit HARD gate (WAC 314-55-095) — logged override only"
`:34`	imports `enforceSalesLimitForSale`
`:171–172`	**builds `{category, quantity}` limit lines from STORED order lines**
`:193`	`enforceSalesLimitForSale(check.limitLines, ...)`
`:207`	the refusal string shown to staff

Note `:171` builds lines from the **stored** order snapshot, not from live menu data. Whatever potency field we add must therefore be **snapshotted onto the order line at placement**, or this gate will evaluate an online order as if no product were low-THC and wrongly block a legal 40-can pickup. This is the subtlest trap in the slice.

3.9 Back office — everywhere the four buckets are shown or edited

Anchor	What
`src/app/admin/compliance/sales-limits/page.tsx:76`	prose: "*1 oz flower, 7 g concentrate, 16 oz solid edible, 72 oz liquid*"
`:84`	prose: "*3 oz / 21 g / 48 oz / 216 oz*"
`:108–110`	the "Liquid edible" summary tile
`:154`	`LIMIT_BUCKETS.map(...)` — the reference table **loops, so it picks up a 5th automatically**
`:158`	`const fmt = (g) => isConc ? "${g} g" : "${gramsToOunces(g)} oz (${g} g)"` — **a mg bucket would render as "7.143 oz"**
`:266–271`	the `rec_liquid` input
`:310–315`	the `med_liquid` input
`:323`	the per-category grams-per-unit textarea
`src/app/admin/compliance/sales-limits/actions.ts:54`	`liquid_edible: num(formData, "rec_liquid", 2016)`
`:63`	`liquid_edible: num(formData, "med_liquid", 6048)`
`src/components/admin/admin-nav-data.ts:181`	nav entry "Sales Limits" → `/admin/compliance/sales-limits`, permission `settings.manage`
`src/lib/compliance/compliance-health.ts:32`	imports settings + overrides
`:286`	`readSalesLimitHealth()` — the health dashboard's limit posture
`src/lib/regulatory/compliance-surface.ts:35`	area key `sales-limits`
`:39`	prose: "*1 oz flower, 16 oz solid edible, 72 oz liquid, 7 g concentrate*"
`src/lib/ai/kb/seed.ts:878`	`REC_LIQUID_OZ = gramsToOunces(RECREATIONAL_LIMITS.liquid_edible) // 72` — **the customer-facing AI concierge quotes this**
`src/lib/medical/tax.ts:217`	`liquidGrams: 216 * METRIC_GRAMS_PER_OUNCE`
`:224`	`liquidGrams: 72 * METRIC_GRAMS_PER_OUNCE`
`src/lib/medical/purchase-limit-display-core.ts:44–46`	the public "Cannabis-infused liquid" row
`src/components/medical/MedicalProgramContent.tsx:65`	`purchaseLimitRows()` — **the public medical-program page**
`docs/COMPLIANCE_BIBLE.md:139`	table row: *"Liquid-form infused
`docs/PRODUCT_NAMING_CONVENTION.md:182`	limits summary line
`docs/POS_FRONTEND_RESEARCH.md:245`	limits summary line

No CCRS surface. `src/lib/ccrs/` contains no sales-detail writer and no limit-bucket field exists in any CCRS payload. Confirmed by grep, not assumed.

3.10 Existing tests that will need updating

Anchor	What
`tests/compliance/sales-limits.test.ts:29`	`expect(RECREATIONAL_LIMITS.liquid_edible).toBe(72 * GRAMS_PER_OUNCE)`
`:35`	`expect(MEDICAL_LIMITS.liquid_edible).toBe(216 * GRAMS_PER_OUNCE)`
`:64–65`	`categoryToBucket("edible-liquid") === "liquid_edible"`
`:99–102`	"*73 liquid-edible units block the 72 oz bucket*" — **stays true for normal liquids, must stay green**
`tests/compliance/cart-limit-meter-core.test.ts`	website meter mirror
`tests/compliance/pos-sale-flow-core.test.ts`	register judge mirror

`tests/compliance/variant-grams-core.test.ts`	per-unit grams
`tests/compliance/pure-selftests.test.ts`	central registry

---

4. The data question — where does per-unit THC mg come from?

This is the scope driver, and the answer is **better than expected**.

4.1 The fields already exist

Migration `0138_structured_product_facts.sql` (Program 3 / Slice 54) already added, to **both** tables:

Anchor	Column
`0138:21`	`inventory_lots.servings_per_pack numeric`
`0138:22`	**`inventory_lots.mg_per_serving numeric` — "mg THC per serving"**
`0138:23`	`inventory_lots.package_thc_mg numeric` — package TOTAL
`0138:29`	`inventory_lots.fact_provenance jsonb`
`0138:35`	`menu_items.servings_per_pack numeric`
`0138:36`	**`menu_items.mg_per_serving numeric`**
`0138:37`	`menu_items.package_thc_mg numeric`
`0138:42`	`menu_items.fact_provenance jsonb`

And there is a **whole pipeline** that populates them, with provenance and an owner-review step:

Anchor	What
`src/lib/inventory/fact-extraction-core.ts:74`	`servingsTimesDose: {servings, mgPerServing}` — parses "10pk 10mg" style facts
`src/lib/inventory/reprocess-core.ts:184–187`	fills `mg_per_serving` **only when `confidence === "verified"`**, records the source
`src/lib/inventory/reprocess-store.ts:77`	selects `mg_per_serving` from `inventory_lots`
`:116`	selects `mg_per_serving` from `menu_items`
`src/lib/pos/transform.ts:79`	`mgPerServing: number | null` on the card type
`:1097–1099`	sets it from the verified exam, records provenance
`:1149`	**emits `mgPerServing` on the published menu card**
`src/app/admin/menu-imports/[id]/facts/page.tsx:430`	**`<FixField label="Mg per serving" name="mgPerServing">` — the owner can already correct it by hand**
`src/app/admin/menu-imports/actions.ts:261`	the save mapping

So Michael already has a back-office screen where he types in mg per serving, it is already stored with provenance, and it already rides onto the published menu card.

4.2 The gap

Grep for `mgPerServing` across `src/` returns **eleven files, all of them intake or import**. Zero components. Zero compliance modules. The fact is **captured and then dropped**:

- `src/app/api/pos/menu/route.ts` (§3.5) does **not** put `mgPerServing` on the POS card. The register never sees it.
- `src/components/cart/CartProvider.tsx:66` (`CartItemInput`) has no potency field. The website cart never sees it.
- `src/lib/pos/import-commit-core.ts:260` writes **`mgPerServing: null`** on commit — worth confirming whether this is a genuine gap in one import path or an intentional default that a later reprocess fills.

So the work is **plumbing an existing, owner-verified fact through three more hops**, not inventing a new data model. That is a materially smaller and safer slice than it first appeared.

4.3 Why `mg_per_serving` alone is not sufficient

The statute's trigger is "*packaged in individual units containing no more than four milligrams*". `mg_per_serving` is a **serving**, and a serving is not always a unit — a 4-pack of 4 mg cans has `servings_per_pack = 4, mg_per_serving = 4`, and each can is one unit at 4 mg, which qualifies. But a single 16 oz bottle labelled "4 servings × 4 mg" is **one unit containing 16 mg**, which does **not** qualify. Same two numbers, opposite legal answers.

Deriving the answer from `servings_per_pack` and `mg_per_serving` would therefore be a **guess**, and the standing rule forbids it. This is precisely why Fix B option 3 from Slice 15 — an explicit per-product flag the owner sets — is the right pattern here. The system may *suggest* the classification from the facts it has, but a human confirms it, and the confirmation is what the register enforces. That also matches the WAC exactly: the qualification is a property of the packaging, which a person can read off the label in two seconds and no parser can infer reliably.

---

5. The plan

Proposed as one slice with a clear internal order. **Not started — awaiting Michael's confirmation of the 200 mg correction in §0.**

Step 1 — make the engine unit-aware (`sales-limits-core.ts`)

Add a fifth bucket `low_thc_liquid` to `LimitBucket` `:43` and `LIMIT_BUCKETS` `:45`. Adding it will **break the build at four literal-enumeration sites** (`:162`, `:265`, `:285`, `:315`) — that is the point, TypeScript enumerates the work for us.

Introduce an explicit unit on the bucket so nothing can silently mix grams and milligrams:

- a `LIMIT_BUCKET_UNITS: Record<LimitBucket, "g" | "mg_thc">` map;
- rename-by-addition on `BucketUsage` `:223` — keep `usedGrams`/`maxGrams` for the four existing buckets so no consumer breaks, and add `unit`, `used`, `max` as the unit-aware fields new code reads. Migrating consumers one at a time is safer than a big-bang rename across ten files.
- `evaluateCart()` `:315` sums the new bucket in **mg THC** (`unitThcMg × quantity`), never grams.
- the reasons string at `:315` must branch on unit — "*200 mg THC*", never "*7.143 oz*".

Constants: `RECREATIONAL_LIMITS.low_thc_liquid = 200` and **`MEDICAL_LIMITS.low_thc_liquid = 200`** (§1.5 — identical, deliberately not ×3), with a comment citing WAC 314-55-095(2)(d) verbatim so no one "corrects" it later.

`LimitCartLine` `:216` gains `unitThcMg?: number | null` and `lowThcLiquid?: boolean`.

Routing in `evaluateCart`: a line whose category maps to `liquid_edible` **and** whose `lowThcLiquid` flag is true routes to `low_thc_liquid` and contributes mg; otherwise it stays in `liquid_edible` and contributes grams. Mutually exclusive, per §1.

Safe default: a product with no flag set is treated as a NORMAL liquid. Unknown must never unlock the more permissive path.

Step 2 — the per-product flag (migration + back office)

New migration `0146_low_thc_liquid.sql` (next free number — verify at build time):

- `menu_items.low_thc_liquid boolean` and `inventory_lots.low_thc_liquid boolean` (nullable = "not yet reviewed"), plus `unit_thc_mg numeric` for the **per-unit** figure that §4.3 shows we cannot derive;
- `sales_limit_settings.rec_low_thc_liquid_thc_mg numeric(10,3) not null default 200` and `med_low_thc_liquid_thc_mg numeric(10,3) not null default 200`;
- no change to `sales_limit_events` — `buckets` is already `jsonb` (`0045:59`).

Back office: extend the facts screen at `src/app/admin/menu-imports/[id]/facts/page.tsx:430` with a "Low-THC beverage (≤4 mg per unit)" toggle and a "THC mg per unit" field, sitting right beside the existing mg-per-serving control, with the WAC citation in the helper text. Extend the save mapping at `src/app/admin/menu-imports/actions.ts:261`.

Add the back-office **list** Michael asked for in Slice 15 Fix B option 3: liquid products awaiting classification, so nothing sits in the safe-but-restrictive default unnoticed.

Step 3 — the settings page

`src/app/admin/compliance/sales-limits/page.tsx`: the reference table at `:154` already loops `LIMIT_BUCKETS`, so the row appears automatically — but `fmt` at `:158` must become unit-aware or it will print "7.143 oz". Add the rec/med inputs beside `:266–271` / `:310–315`, and correct the prose at `:76` and `:84`. Extend `actions.ts:54/:63`. Extend `clampLimitProfile` so the owner can tighten 200 mg but never widen it, exactly as AN-2 does today.

Step 4 — plumb the fact to the register

`src/app/api/pos/menu/route.ts:169`: attach `unitThcMg` and `lowThcLiquid` to the POS card beside `unitGrams`. Extend the card type at `sale-flow-core.ts:80`, the priced line at `:285`, and **`limitLinesFor()` at `:406`** to carry them through. The meter at `SaleFlow.tsx:2374` then works with no change to the call site.

Step 5 — plumb the fact to the website

Extend `CartLimitLineInput` at `cart-limit-meter-core.ts:43` and `cartLimitLines()` at `:65`. Extend `CartItemInput` at `CartProvider.tsx:66` and every `addItem` call site. Both hard blocks then fix themselves through `cartLimitBlock()` at `:122`. `CartLimitMeter.tsx:70` needs the unit-aware label so the shopper sees "*120 mg of 200 mg*", not a fake ounce figure.

Step 6 — the order snapshot (§3.8 trap)

Persist `low_thc_liquid` / `unit_thc_mg` onto the stored order line at placement (`src/app/api/orders/route.ts:117`), and read them back at `completion-gate.ts:171`. Without this, every online low-THC order is re-evaluated at pickup as a normal liquid and wrongly blocked.

Step 7 — the truth surfaces

Update the AI concierge seed (`kb/seed.ts:878`), the compliance surface prose (`compliance-surface.ts:39`), the public medical table (`purchase-limit-display-core.ts:44`), `COMPLIANCE_BIBLE.md:139`, and the two doc summary lines. Fix the stale header comments at `sales-limits-core.ts:12–13`.

Step 8 — tests, RED first

Every one of these gets written and **demonstrated failing** before the fix:

1. `RECREATIONAL_LIMITS.low_thc_liquid === 200` and unit is `mg_thc`.
2. **`MEDICAL_LIMITS.low_thc_liquid === 200` — the anti-×3 guard.** Its failure message will name WAC 314-55-095(2)(d) explicitly.
3. 50 × 4 mg units passes; 51 × 4 mg blocks at 204 mg.
4. 100 × 2 mg units passes (200 mg exactly, at the line).
5. A flagged low-THC line contributes **zero** to `liquid_edible` — the mutual exclusivity of (E) and (F).
6. An **unflagg**
